



**Politecnico
di Torino**

Runtime Security Monitoring System basato su eBPF

Pipeline end-to-end e implementazione hook

Simone Romano · Giuseppe Insalaco

Riferimento Progettuale e Vincoli Architeturali



Modello Tracee

- General-purpose, kernel moderni
- Ring buffer principale
- Policy engine complesso



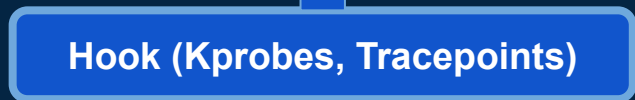
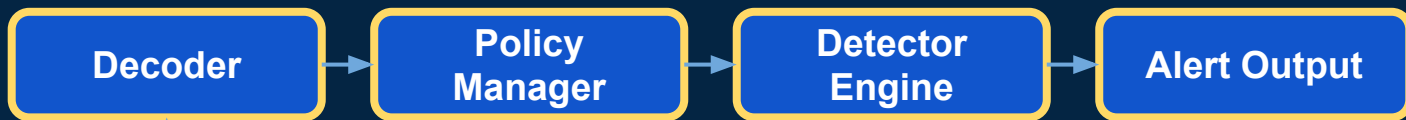
Nostro Approccio (Rocky Linux 4.18)

- Mirato e controllabile
- Limiti del verifier 4.18
(no bpf_task_pt_regs)
- Fallback su perf buffer
- Policy layer essenziale

Impatto Verifier: Necessità di usare offset dimostrabili e slot fissi per superare le restrizioni del kernel

Architettura e Pipeline End-to-End

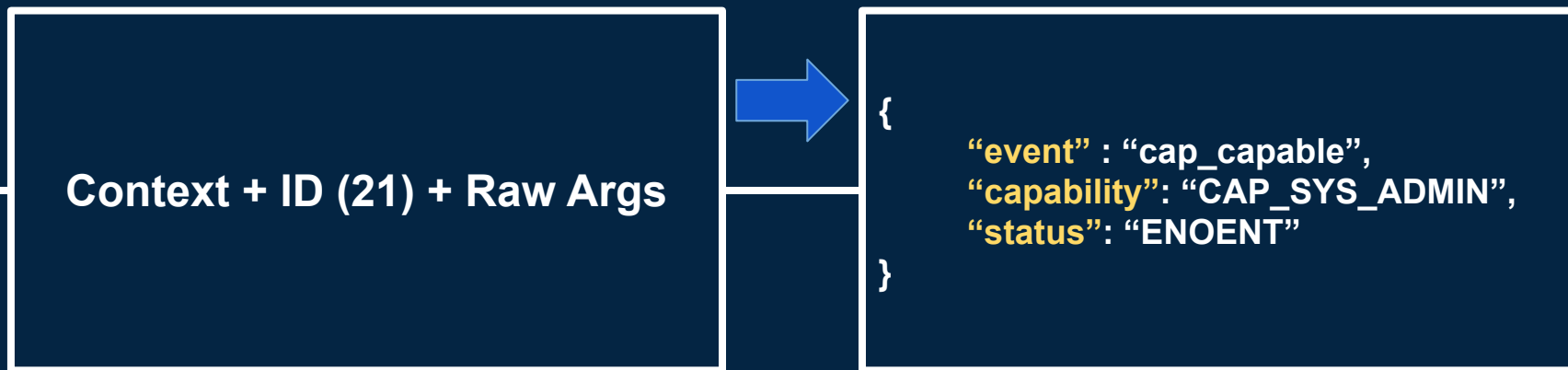
Userspace



Kernel Space

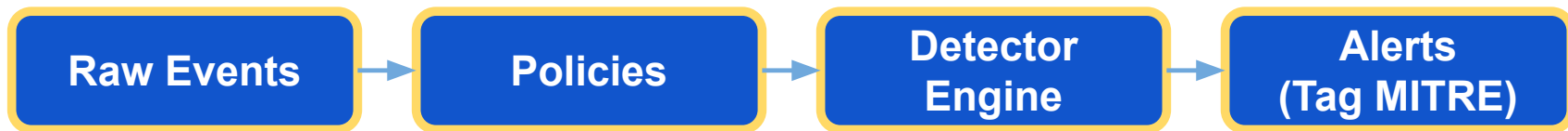
- **Acquisizione:** Selezione e attach dinamico dei soli programmi richiesti.
- **Trasporto:** Perf buffer come canale operativo primario
- **Elaborazione:** Decodifica in Go, normalizzazione e filtraggio

Event Model e Normalizzazione



- Contratto Binario: Registry condiviso per mappare ID eBPF a schemi Go.
- Decodifica Tipizzata: Supporto per scalari, array di stringhe e puntatori.
- Normalizzazione: Conversione di capability, mask ed errno in formati leggibili.

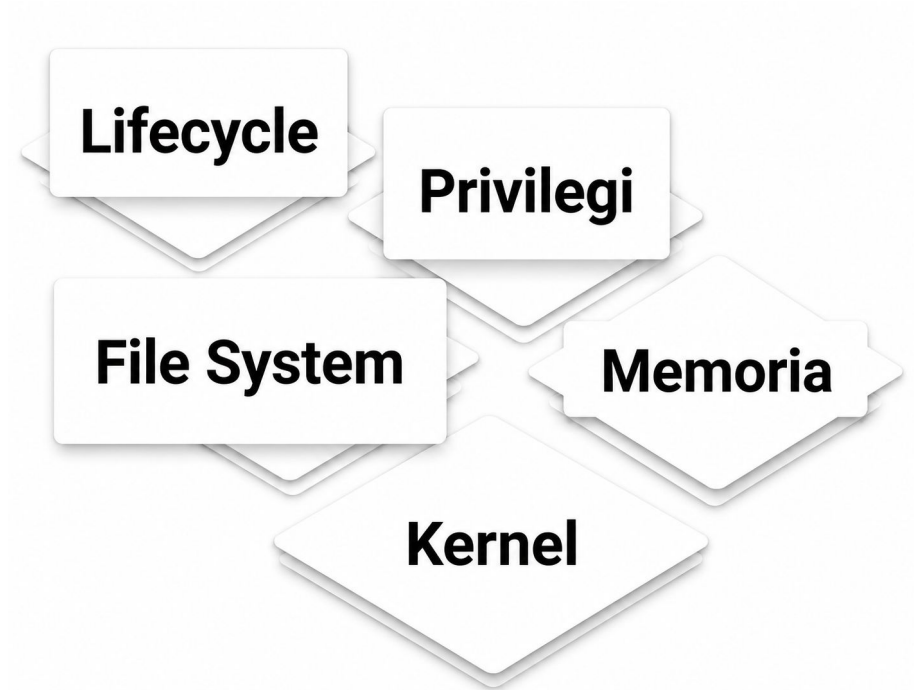
Layer di Policy e Detection (YAML)



- **Logica Dichiarativa:** Policy e detector definiti in YAML, validati al caricamento.
- **Astrazione:** I detector valutano gli argomenti senza conoscere i dettagli di decodifica (event.ArgString).
- **Stato e Mapping:** Supporto per detector stateful (finestre brevi) e mapping MITRE ATT&CK.

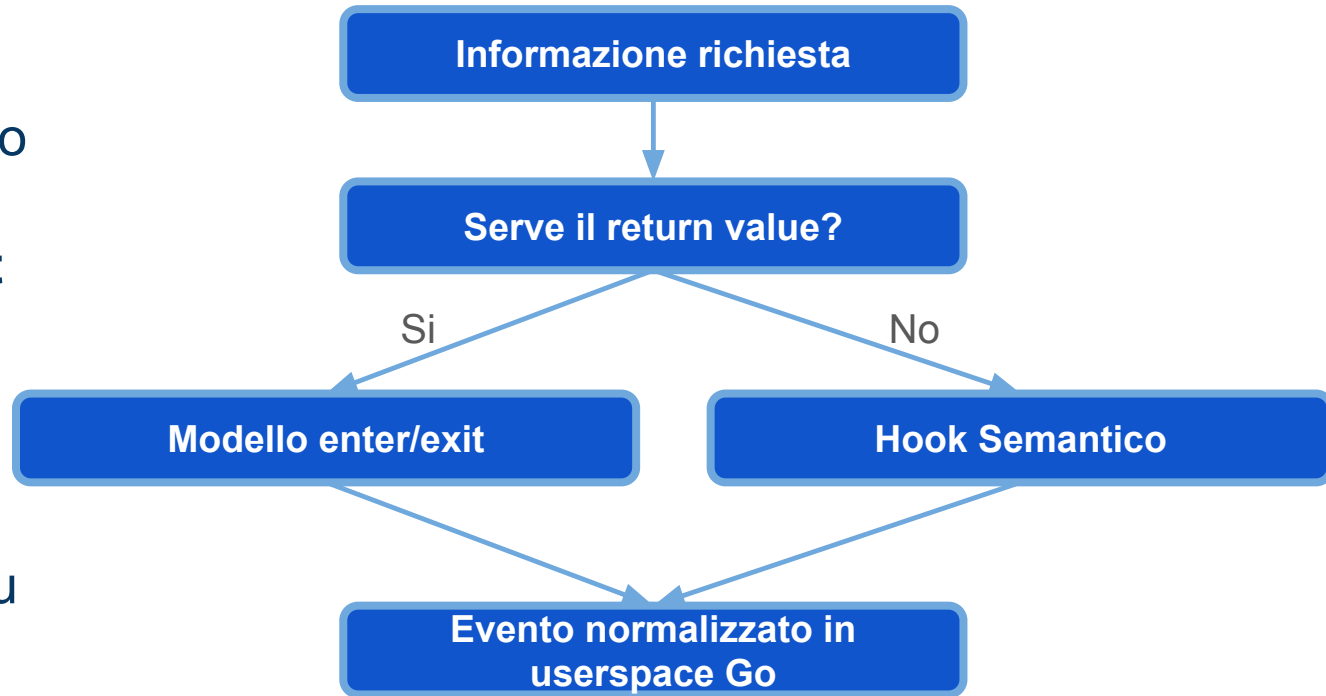
Focus: Superficie Process, Security e Kernel

- **Filosofia di Copertura:** Prediligere hook semantici (kprobe/security) rispetto a tracepoint generici rumorosi.
- **Controllo del Ritorno:** Modello enter/exit applicato selettivamente per intercettare gli esiti delle syscall.
- **Obiettivo:** Monitoraggio mirato delle minacce host-level, escludendo il layer di rete



Strategia di Implementazione degli Hook

- **Hook Semantici**
quando il kernel
espone già l'oggetto
utile
- **Modello enter/exit**
quando serve
conoscere il return
value
- Adattamento ai
vincoli del verifier su
Rocky Linux 4.18



Exec Monitoring: Tentativi, Successi e Fallimenti

 Hook	 Finalità di Sicurezza
<code>sched_process_exec</code>	Conferma l'exec riuscita
<code>execve / execveat</code>	Intercettano il tentativo di esecuzione (inclusi dirfd e flag)
<code>process_execute_failed</code>	Isola i tentativi falliti o negati
<code>security_bprm_check</code>	Osserva la fase security del caricamento binario.

Privilegi, Identità e Controllo Capability

 Hook	 Finalità di Sicurezza
setuid/setgid family	Richieste di cambio UID/GID e varianti
security_task_fix_setuid	Transizioni UID/EUID nel percorso security
commit_creds	Applicazione finale delle nuove credenziali
cap_capable, ptrace, prctl	Capability check e manipolazione processi

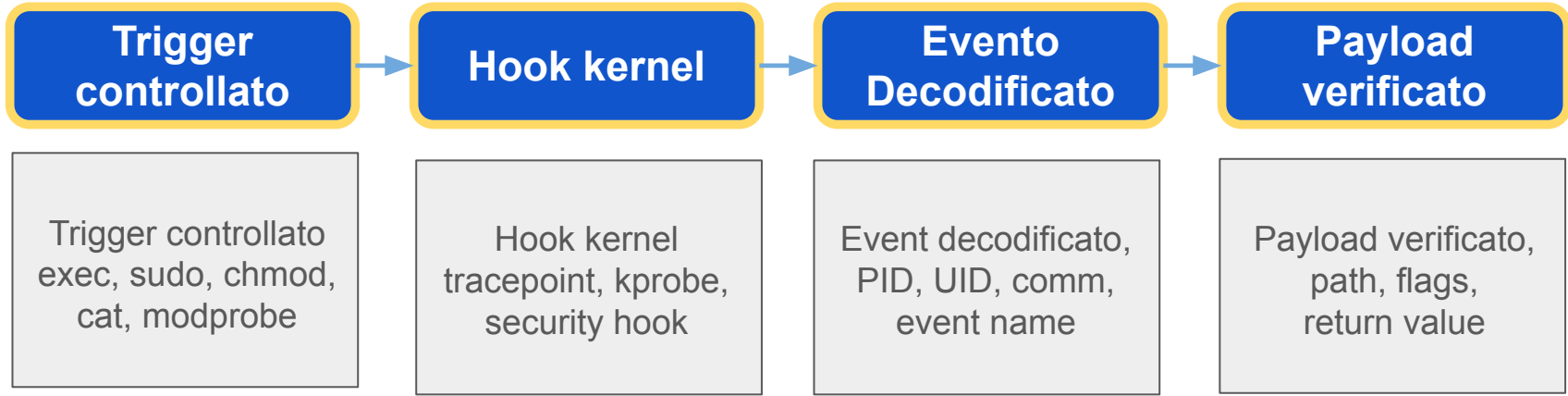
Sicurezza File, Permessi e Filesystem

 Hook	 Finalità di Sicurezza
security_file_open / open	Accesso a file sensibili con path e metadata
chmod / chown family	Modifica di permessi e ownership con esito finale
security_inode_unlink / rename	Cancellazioni e rename potenzialmente sospetti
security_sb_mount / umount	Mount, bind mount e ambienti filesystem evasivi

Memoria, Namespace e Kernel Hardening

 Hook	 Finalità di Sicurezza
process_vm_writev / process_vm_readv	Accesso diretto alla memoria di processi terzi
mmap / mprotect / memfd_create	Fileless execution e memoria eseguibile
setns / unshare / switch_task_ns	Movimenti tra namespace e tentativi di container escape
do_init_module / module_load	Caricamento moduli kernel e potenziali rootkit
security_bpf / register_kprobe / proc_create	Abuso eBPF, kprobe dinamiche e superfici procfs

Testing e Validazione su Rocky Linux 4.18



- Validazione eseguita direttamente sulla VM target Rocky Linux 4.18
- Controllo base di CPU usage e rumore operativo durante le run

Limiti Rilevati

- I detector attuali identificano principalmente point anomaly.
- Alcuni hook process/security sono ancora rumorosi e richiedono filtri più mirati.
- Non tutti gli hook espongono il return value, quindi in alcuni casi osserviamo il controllo security ma non l'esito finale.
- La copertura non-network è ampia ma non completa: restano hook avanzati process/security e kernel tampering da valutare.

Step Tecnici Pianificati

- Transizione verso collective anomalies (correlazione stato temporale).
- Implementazione di dedup e throttling per la gestione degli alert.
- Benchmark di carico su CPU.
- Introdurre kernel-side filtering minimo per ridurre rumore e overhead.
- Usare il modello enter/exit sugli hook dove il return value è essenziale.
- Estendere la copertura con hook process/security avanzati compatibili con Rocky Linux 4.18.

PLACEHOLDER PER PARTE DI GIUSEPPE